



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

INGENIERÍA DE SOFTWARE II



ACTIVIDAD I PRODUCTO IV:

“Risk Storming Simulado para Fintech Mobile (Serverless + API IA)”

Docente:

Félix Marquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

1. Contexto de Arquitectura y Técnica de Risk Storming

La aplicación es una plataforma de evaluación crediticia de móvil que se ejecuta mediante una arquitectura Serverless (Proveedor Único de Nube) y utiliza un modelo de Machine Learning/ Large Language Model (LLM) presentado como API de terceros para calcular el riesgo crediticio . Para realizar el Risk Storming, todos los roles (Desarrolladores , Arquitectos , Pruebas , Seguridad e Ingeniero de Producto) participaron en una reunión virtual sobre la arquitectura para identificar y colocar amenazas; en lugar de completar una lista de impactos , los participantes colgaron adhesivos con amenazas en los componentes Arquitectónicos (Aplicación móvil, API Gateway , Funciones Serverless , Base de datos y API externa de IA).

2. Identificación Detallada de los 4 Riesgos por Categoría

- **(a) Técnico / Deuda Técnica (Acoplamiento a API Externa de IA):**
 - **Descripción:** Diseñar las funciones *Serverless* del backend acopladas directamente a las estructuras de datos JSON específicas de la API del proveedor de IA, sin emplear un patrón de arquitectura hexagonal o de adaptación (*Adapter Pattern*).
 - **Impacto en el Negocio:** Si el proveedor actualiza o deprecia la versión de sus contratos de API, cualquier *breaking change* romperá en tiempo de ejecución las funciones *Serverless* en producción. Corregirlo requerirá un esfuerzo masivo de refactorización urgente en todo el código base, ralentizando la velocidad de entrega de nuevas funcionalidades.
- **(b) Ciberseguridad / Cadena de Suministro (Ataque a Paquetes de Terceros - Supply Chain):**
 - **Descripción:** Inyección de paquetes o dependencias maliciosas (*Typosquatting*, *Dependency Confusion* o repositorios comprometidos) dentro del código de Node.js/Python ejecutado por las funciones *Serverless*.

- **Impacto en el Negocio:** Un atacante podría interceptar las ejecuciones del backend y exfiltrar credenciales de acceso a la base de datos o datos sensibles (cédula, ingresos, fotos de documentos) de los solicitantes de crédito. Este riesgo es crítico en arquitecturas *Serverless* donde se instalan habitualmente decenas de módulos ligeros de repositorios públicos (*npm*, *PyPI*).
- **(c) Vendor Lock-in / Nube (Dependencia Exclusiva de un Proveedor Serverless):**
 - **Descripción:** Utilizar arquitecturas y servicios 100% propietarios del proveedor de nube (como *AWS EventBridge*, *DynamoDB*, *Cognito* o *AWS Step Functions*) de forma directa en la lógica de negocio sin una capa de abstracción.
 - **Impacto en el Negocio:** Si el proveedor incrementa sus tarifas por invocación de funciones *Serverless* o lecturas en base de datos, o si experimenta caídas prolongadas de servicio en la región, la fintech quedará atrapada. Migrar la infraestructura a otro proveedor (ej. Google Cloud o Azure) tomaría meses de trabajo técnico e inversión financiera no planificada.
- **(d) Ético o Legal / IA (Sesgo Algorítmico y Falta de Explicabilidad):**
 - **Descripción:** Operar con un modelo de IA de terceros "caja negra" (*black box*) que presenta sesgos demográficos no controlados o sufra de alucinaciones en el cálculo del score crediticio.
 - **Impacto en el Negocio:** Rechazos discriminatorios o injustificados de solicitudes de crédito a ciertos colectivos. Esto expone a la fintech a severas sanciones regulatorias por entes financieros, auditorías legales por falta de derecho a explicación del rechazo (*Right to Explanation* bajo normativas de protección de datos) y pérdida irrecuperable de reputación de marca.

3. Análisis Cualitativo del Riesgo Crítico Selección del Riesgo: (b) *Ciberseguridad y Cadena de Suministro (Inyección en dependencias Serverless).*

- **Justificación de Selección:** Aunque los riesgos legales de la IA son altos, el compromiso de la cadena de suministro en un entorno bancario/fintech causa un daño operativo e inmediato destructivo (robo masivo de datos o compromiso total del entorno cloud).
- **Evaluación Cualitativa (Escala 1 a 5):**
 - **Probabilidad (P): 4 / 5 (Alta):** La tasa de ataques a librerías de software libre (npm/pip) y la vulnerabilidad de dependencias transitivas han crecido exponencialmente en la industria de desarrollo web y móvil.
 - **Impacto (I): 5 / 5 (Muy Alto / Catastrófico):** El filtrado de datos bancarios de los clientes ocasiona paralización del servicio, multas legales severas y potencial cierre de operaciones.
- **Cálculo del Risk Score:**

Risk Score = Probabilidad • Impacto = 4 • 5 = 20 (*Zona Crítica / Prioridad Máxima*) [cite: 1]

4. Estrategia de Respuesta y Traducción Tangible al Sprint Backlog

- **Estrategia Elegida: Mitigación** (Implementar controles técnicos y de automatización en el flujo de integración continua para reducir la probabilidad de despliegue de paquetes maliciosos a casi cero).
- **Traducción al Product / Sprint Backlog:** En lugar de documentar este riesgo únicamente en un archivo PDF o hoja de cálculo, el equipo de desarrollo crea una **Historia de Habilidad de Seguridad (Security Enabler Story)** para asignarle tiempo de trabajo en el siguiente Sprint:

[SECURITY ENABLER] Integración de Análisis Automático de Dependencias (SCA) y Generación de SBOM en Pipeline CI/CD

- **ID:** SEC-104

- **Tipo:** *Enabler / Technical Debt Mitigation*
- **Prioridad:** Máxima (Bloqueante para el despliegue a producción)
- **Estimación:** 5 Puntos de Historia (1 Sprint)

Descripción / Historia de Usuario: Como *Ingeniero de Ciberseguridad y DevOps*, quiero integrar herramientas de análisis estático de dependencias (*Software Composition Analysis - SCA*) y un generador de *Software Bill of Materials* (SBOM) dentro del pipeline de CI/CD de las funciones *Serverless*, para prevenir la inclusión de librerías con vulnerabilidades conocidas (CVE) o licencias incompatibles antes de cada despliegue.

Criterios de Aceptación Tangibles:

1. **Escaneo Automático:** Configurar *Snyk* u *OWASP Dependency-Check* en la etapa de pre-compilación de GitHub Actions / GitLab CI.
2. **Uso de Build Break:** Si el análisis SCA detecta alguna dependencia de terceros con una vulnerabilidad clasificada como *High* o *Critical* (según el estándar CVSS), el pipeline debe abortar el *build* automáticamente y notificar al canal del equipo.
3. **Generación de SBOM:** Exportar automáticamente en cada compilación de producción un archivo de manifiesto *SBOM* en estándar CycloneDX o SPDX con la lista detallada de componentes, versiones y firmas digitales de software utilizado.
4. **Fijación de Versiones (*Locking*):** Configurar reglas estricta en *package-lock.json* o *poetry.lock* prohibiendo la resolución dinámica de versiones secundarias sin revisión explícita del equipo.